

Question description

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1, 1)$, and the lower-right square has coordinates (n, n) .

Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Constraints

- $1 \leq n \leq 1000$
- $1 \leq q \leq 2 \cdot 10^5$
- $1 \leq y_1 \leq y_2 \leq n$
- $1 \leq x_1 \leq x_2 \leq n$

Input

The first input line has two integers n and q : the size of the forest and the number of queries.

Then, there are n lines describing the forest. Each line has n characters: `.` is an empty square and `*` is a tree.

Finally, there are q lines describing the queries. Each line has four integers y_1, x_1, y_2, x_2 corresponding to the corners of a rectangle.

Output

Print the number of trees inside each rectangle.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) <pre>4 3 .*. *.* **.</pre> 2 2 3 4 3 1 3 1 1 1 2 2 EXPECTED OUTPUT <pre>3 1 2</pre>	INPUT (STDIN) <pre>4 3 .*.* *.* **.* ***.</pre> 1 2 3 4 3 2 3 4 1 4 2 3 EXPECTED OUTPUT <pre>6 2 0</pre>

▼ Mandatory Test Cases

Test Case 1
KEYWORD
<code>for(i=1;i<=n;i++)</code>

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
8

Test Case 2
TOKEN COUNT
310

Test Case 3
NLOC
28

```
1 #include <stdio.h>
2
3 #define MAXN 1005
4
5 int pref[MAXN][MAXN];
6 char row[MAXN];
7
8 int main() {
9     int n, q, i, j;
10    if (scanf("%d %d", &n, &q) != 2) return 0;
11    for(i=1;i<=n;i++) {
12        scanf("%s", row);
13        for (j = 1; j <= n; j++) {
14            int val = (row[j - 1] == '*') ? 1 : 0;
15            pref[i][j] = pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1] + val;
16        }
17    }
18    for (int k = 0; k < q; k++) {
19        int y1, x1, y2, x2;
20        if (scanf("%d %d %d %d", &y1, &x1, &y2, &x2) == 4) {
21            if (y1 > y2 || x1 > x2) {
22                printf("0\n");
23            } else {
24                int ans = pref[y2][x2] - pref[y1 - 1][x2] - pref[y2][x1 - 1] + pref[y1 - 1][x1 - 1];
25                printf("%d\n", ans);

```

```
26     }  
27     }  
28 }
```

```
29     return 0;  
30}  
31
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4 3

. * . .

* . **

** . .

2 2 3 4

3 1 3 1

1 1 2 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree13.exe

3

1

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 3

. * . *

* . **

** . *

*** .

1 2 3 4

3 2 3 4

1 4 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree13.exe

6

2

0

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>